Guided Tutorial: Adversarial Search in AI

Learning Outcomes

By the end of this tutorial, students will be able to:

1. Understand the concept of adversarial search and its role in AI game-playing.
2. Apply the Minimax algorithm for decision-making in competitive environments.
3. Implement Alpha–Beta pruning to optimize search.
4. Solve and analyze a small-scale game scenario.

Activity 1: Introduction to Adversarial Search

Adversarial search is used in multi-agent environments where agents (players) compete with each other — typically zero-sum games like chess or tic-tac-toe.

In such games:

- One player's gain is the other player's loss.
- The search tree alternates between MAX (our turn) and MIN (opponent's turn).

Minimax Algorithm

Idea:
- MAX tries to maximize the final score.
- MIN tries to minimize it.

Algorithm steps:
1. Generate the game tree to possible moves.
2. Apply a utility value (score) at terminal nodes.
3. Propagate values upward: MAX node → choose the maximum; MIN node → choose the minimum.

Formula:

$$Minimax(n) = \begin{cases} Utility(n), if\ n\ is\ terminal \\ max(Minimax(c))\ for\ children(c)\ if\ MAX\ node \\ min(Minimax(c))\ for\ children(c)\ if\ MIN\ node \end{cases}$$

Task 1

Question:

Explain in your own words how adversarial search differs from standard search (like A* or BFS). Then, identify:
- Who is MAX and who is MIN in a Tic-Tac-Toe game?
- Why is it called adversarial?

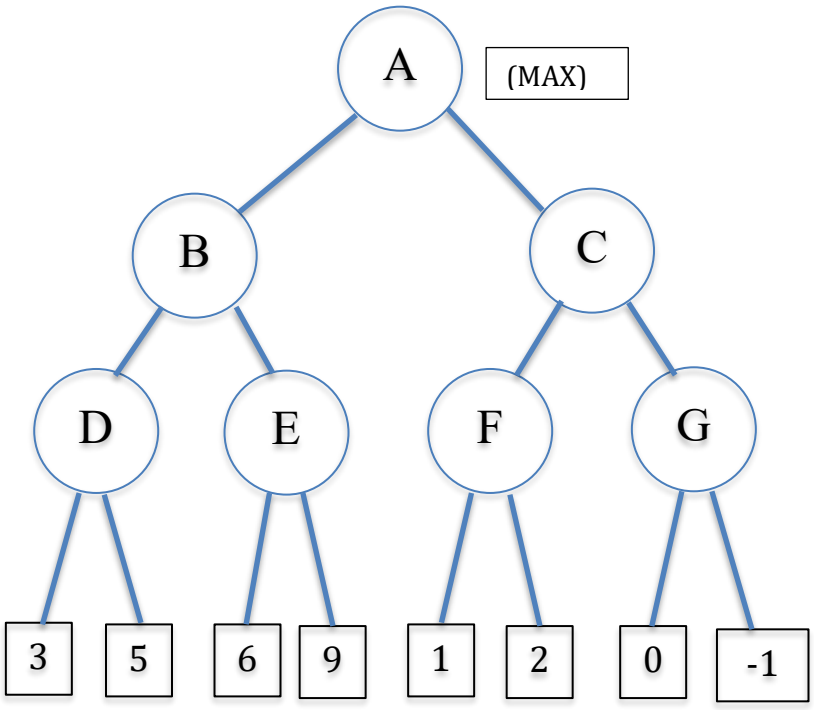
Expected answer summary:
Adversarial search involves two or more competing agents, while standard search assumes a single agent trying to reach a goal. In Tic-Tac-Toe, the player (X) is MAX trying to win, and the opponent (O) is MIN trying to block or minimize X's chances. It's adversarial because each move by one affects the strategy of the other.

Activity 2: Alpha–Beta Pruning Example

Alpha–Beta pruning improves Minimax by eliminating branches that cannot possibly influence the final decision.

- α (alpha): Best value that MAX can guarantee so far.
- β (beta): Best value that MIN can guarantee so far.
- If at any point α ≥ β, we prune that branch — it won't affect the result.

Example Game Tree:



Step-by-Step Solution (with Alpha–Beta pruning)

1. Start at A (MAX): $\alpha = -\infty$, $\beta = +\infty$
2. Go to B (MIN): visit D (MAX) → MAX = 5, update B: $\beta = 5$
   Visit E (MAX): first child 6 → prune E's 9 (since $6 \geq 5$), B returns 5
3. A updates $\alpha = 5$
4. Go to C (MIN): visit F (MAX) → MAX = 2, update C: $\beta = 2$
   Since $\beta \leq \alpha$ ($2 \leq 5$), prune G's branch.
5. A chooses max(5, 2) → Final decision = 5

Optimal value = 5
Pruned branches: E's (9) and G's subtree.

Task 2

Draw or explain the pruning process for the above tree, showing which branches were skipped. Then, describe in one sentence how Alpha–Beta pruning improves Minimax efficiency.

Expected answer summary:
Alpha–Beta pruning skips nodes that cannot influence the final decision, thus reducing the number of evaluated states without changing the result.